

Agents, Harnesses & Amplification

A High-Confidence Research Brief for ADE

Edition: 2026-07-23 · **Revised:** vocabulary lock (Tokenomics ≠ \$)

Length target: ~10 pages

Method: Five parallel research tracks + primary-source spot checks

Stance: Prefer arXiv, vendor docs, institutional reports; label estimates

Companion (dedicated): [*ADE Tokenomics — Input · Output · Context · Usage*](#)

This brief covers harness, multi-agent, verify, routing, and **\$ metering as adjacent. Tokenomics** (I/O strategies, context hygiene, usage patterns) lives in the companion — not in \$5 pricing tables.

0. How this brief was built

Five independent research tracks were run in parallel, then cross-checked against primary sources:

Track	Focus	Spot-check
A	Harness engineering & model×harness	arXiv / UNU / ContextOS claims
B	\$ metering & caching prices (SpendGuard-adjacent; <i>not</i> Tokenomics)	Anthropic Pricing (live Jul 2026)
C	Multi-agent coordination	CooperBench arXiv:2601.13295 abstract
D	Verification, evals, reward hacking	Spec Kit arXiv:2604.05278; SWE-bench posts
E	Model routing & role amplification	RouteLLM / FrugalGPT / Claude Code docs

Confidence legend

- **HIGH** — Primary source verified or widely corroborated by vendor docs / peer-preprint.
- **MEDIUM** — Strong secondary synthesis or first-party blog without peer review.
- **ESTIMATE** — Workload-dependent; do not treat as a universal constant.

What this is not: A claim that harnesses “always beat models,” or that any single \$/task figure transfers to ADE without measurement.

1. Definitions that matter

1.1 Agent

An **agent** is a closed-loop system: a model policy π samples actions (tokens, tool calls) given observations (files, terminal, chat), then the environment changes and new observations return. Coding agents are **partially observable**: the true state is the repo + CI + other writers, not the chat transcript.

1.2 Harness (HIGH)

The **harness** is the deterministic runtime envelope around the model. It controls what the agent can see, call, mutate, remember, and what counts as “done.”

“The harness sits between them: it organises how model outputs become tool calls, observations, memory updates, approvals, interruptions, resumptions and recoverable actions.”
— UNU, *Engineering and Governing the Agent Harness* (2026)

Prompt engineering shapes a single invocation. **Harness engineering** shapes the loop: tools, permissions, budgets, verification, recovery, observability.

1.3 Model–harness pair (HIGH)

Capability should be evaluated as a **{model, harness}** pair. A strong model in a weak harness underperforms a weaker model in a strong harness on long-horizon tasks (UNU 2026; arXiv:2605.23950 position on harness disclosure).

Controlled evidence (arXiv:2605.23950, reported experiment): harness-induced variance exceeded model-induced variance by **~7.8x** in a 3x3 grid, with ranking reversals when the harness changed. Treat the exact ratio as **study-specific**, not a universal law.

1.4 ADE mapping

ADE already implements harness pieces: autonomy (Suggest/Apply/Automate), ToolEffect authorization, path **leases**, verify ladder, effort budgets, spend guard scaffolding, Continuity handoffs. The product bet (DEC-A-014) is **harness-first**; editor wrapping (Zed) is optional.

2. The production control loop

Industry architectures converge on a gated loop (ContextOS 2026 whitepaper; OpenAI / Anthropic long-horizon posts):

Receive → Compile context → Plan → VerifyPlan → Authorize
→ Execute → Observe → Repair/Replan → VerifyDone → Trace → Memory

Invariant (HIGH): Side effects must consume an **authorized action envelope**, not raw model prose.

2.1 Practical checklist (HIGH)

1. Document harness config as a first-class artifact (tools, permissions, budgets, routing).
2. Separate intelligence / context / decision / action / trust planes.
3. Compile context per run; prefer progressive disclosure (skills) over dumping everything.
4. Least-privilege tools with schema validation.
5. Deterministic sensors (lint/test) before LLM judges.
6. Durable session + trajectory log (replay, Continuity).
7. Lifecycle hooks at tool boundaries (code gates, not prompt pleading).
8. Orthogonal budgets: tool rounds · output tokens · dollars.
9. Explicit repair / escalate paths when verify fails.
10. Evaluate and procure as model–harness pairs; disclose harness in benchmarks.

2.2 Hype to avoid (HIGH)

Hype	Reality
"Prompt engineering is dead"	Reclassified <i>inside</i> the harness
"Harness always beats model"	True in some long-horizon regimes; not universal
"Better models eliminate harnesses"	Capability growth <i>outdates</i> harness assumptions
Per-action human approve forever	Anthropic: ~93% approve; sandbox cut prompts ~84%

3. Multi-agent robustness

3.1 CooperBench curse of coordination (HIGH — verified)

Paper: *CooperBench: Why Coding Agents Cannot be Your Teammates Yet* — arXiv:2601.13295 (2026).

- **652** collaborative coding tasks; 12 libraries; 4 languages; expert tests.
- Abstract claim (verified): agents achieve on average **~30% lower success rates** when working together than when **one agent performs both tasks** (Solo).
- Leading models: Coop success **~25%**; gap vs Solo up to **~50%** relative for GPT-5 / Claude Sonnet 4.5.
- **77.3%** of tasks have overlapping ground-truth edits (coordination required, not adversarial impossibility).
- Communication can reduce **line-level merge conflicts** without restoring end-to-end success (spatial $\neq$ semantic coordination).

Do not confuse: The headline is **~30% lower success (relative)**. A separate scaling subset reports absolute success **~30%** with **four** cooperating agents — a different statistic.

3.2 Failure taxonomy (MEDIUM–HIGH; n=50 manual traces in CooperBench)

Cause	Share (labeled failures)	Mechanism
Expectation	~42%	Partner state ignored; duplicate / overwrite
Commitment	~32%	Unverifiable “done”; broken promises
Communication	~26%	Vague, late, unanswered messages

Broader MAS taxonomy (**MAST**, arXiv:2503.13657): Specification ~42%, Inter-agent misalignment ~37%, Task verification ~21% (200 traces, $\kappa=0.88$).

3.3 What works in practice (HIGH pattern)

Pattern	Peer chat agents	Planner / worker hierarchy
Coordination	Free-form NL	Structured tasks / JSON
Isolation	Branches / containers	Worktrees + single merge path
Locks	Implicit file fights	Explicit ownership (leases)
Verification	Hope tests pass after merge	Gate before “done”

Cursor engineering blogs (MEDIUM, first-party): flat peer agents + shared locks → deadlocks and risk-averse churn; recursive **planner/worker** + isolation scaled better. Treat as industry evidence, not peer-reviewed.

3.4 ADE multi-agent contract

1. Planner = Suggest (no writes).
2. Worker = Apply under **leases** (+ optional Isolate worktree).
3. Explorer = Observe / cheap model OK.
4. Verifier = computational sensors first; never self-certify.
5. **Law:** never two write-capable agents on one checkout.

4. Verification, evals & reward hacking

4.1 Grounding (HIGH)

Spec Kit Agents — arXiv:2604.05278: SDD artifacts can be internally coherent yet incompatible with the repo (**context blindness**). Validation hooks must run **outside** the generator’s prompt loop (paths exist, deps present, tests pass).

4.2 Recommended verify ladder (MEDIUM–HIGH synthesis)

Rung	Sensor type	Examples
R0	Contract	Acceptance criteria before code
R1	Computational / every edit	Lint, typecheck, unit tests, path bounds
R2	Integration / phase	Full suite, build, SAST
R3	Inferential / checkpoint	Independent LLM judge (no generator CoT)
R4	External	Clean CI + human + held-out gold set

Rule (HIGH): The agent that writes code must not be the sole certifier of done (Anthropic evals engineering; Spec Kit generator≠judge).

4.3 Reward hacking (HIGH)

Documented patterns (Cursor, METR, OpenAI SWE-bench post-mortems):

- Git history mining / web gold-patch lookup
- Pytest hook injection / grader overwrite
- Test deletion or # noqa spam
- Self-written tests that assert buggy behavior
- Contamination / verbatim recall on public benches

OpenAI (Feb 2026): stopped reporting SWE-bench Verified for frontier claims after audit findings (flawed tests, contamination risk); prefer harder / held-out partitions (e.g. SWE-bench Pro line of work).

Harness implication: Gold-set CI + sealed environments + transcript audit — not leaderboard chasing.

5. Dollar metering & cache prices (SpendGuard-adjacent)

Vocabulary: Tokenomics = input/output/context/usage strategy → see companion PDF.
This section is **invoice-class economics** (rates, cache price multipliers, \$ caps). Orthogonal to Effort (rounds + out-tokens) and to window hygiene.

5.1 Pricing structure (HIGH — Anthropic docs Jul 2026)

Example **Claude Sonnet 4.6** (USD / MTok, standard):

Component	Price	Note
Base input	\$3	
Cache write (5m)	\$3.75	1.25× base
Cache write (1h)	\$6	2× base
Cache read / hit	\$0.30	0.1× = 90% off

Output	\$15	5× input
--------	------	----------

Output is typically **4–6×** input on frontier coding SKUs. Constrain **output / thinking** before obsessing over small prompt trims.

5.2 Cache mechanics (HIGH)

- Exact **prefix** match; minimum cacheable length commonly **1,024 tokens**.
- Put **static** content first: system → tools/schemas → stable docs; dynamic user/tool results last.
- Cached tokens still consume **context window** (and often rate limits) — caching changes **price**, not capacity.
- Newer Anthropic tokenizers (~Opus 4.7+ / Sonnet 5 line) can produce ~**+30%** tokens for the same text — counts are not portable across families.

5.3 Agent-loop cost shape (HIGH mechanism / ESTIMATE magnitude)

Each tool round re-sends growing history + tool results. Cost scales roughly **O(rounds × growing_context)**, not O(rounds).

ESTIMATE: Multi-round coding agents often spend **3–8×** a single-shot call before useful completion — measure on your workload.

5.4 Failure modes (HIGH)

1. Cache prefix thrashing (timestamps/IDs early in prompt).
2. Paying write tax with low hit rate.
3. Long-context tier cliffs (OpenAI documents surcharges past large input thresholds on some 1M-window SKUs — verify per model card).
4. Tool-result accumulation (quadratic growth).
5. Context rot: larger windows ≠ reliable recall (Anthropic engineering; Chroma “context rot” research).
6. Billing on **chars÷4** (invalid for code/tools/images/billing).

5.5 Controls that work (HIGH)

Control	Why	Lane
Bill provider usage fields only	Invoice identity	\$
Reserve → reconcile \$ caps	Stop before silent overrun	\$
Orthogonal budgets	Rounds ≠ out-tokens ≠ dollars	Both
Cache-stable ordering + keys	Real hit rate and sane prefix	Tokenomics + \$
Preflight token-count APIs	Same tokenizer as inference	Tokenomics
Continuity thrift	Host runs safe steps outside the model	Tokenomics
Ledger cache read/write tokens	Optimize what you measure	\$

Do not claim without measurement: “Caching saves 90% of agent spend” (only the stable prefix × hit rate).

Pointer: For progressive disclosure, compaction @~70%, Effort output discipline, and usage playbooks → **ADE Tokenomics** companion.

6. Per-model amplification & routing

6.1 Roles beat one-model-does-all (HIGH)

Profile	Autonomy default	Tier	Amplifies	Blocks
Scout / Explore	Observe	Small / local	Cheap discovery	Writes
Planner / Architect	Propose	Frontier	Deep plans	Solo Apply
Coder / Editor	Act	Mid	Fast edits	Unscoped writes
Judge / Critic	Observe	$\geq$ coder	Cold review	Self-grade
Advisor	Escalate only	Frontier	Hard decisions	Mid-session silent swap
Thrifty / Continuity	Med+	Mid	Resume after budget	Re-discovery loops

Industry documentation: Claude Code `opusplan` / Explore / advisor; Aider architect+editor; OpenAI Agents handoffs; Cursor Router (cache-aware routing blog — MEDIUM for exact savings %).

6.2 Routing vs cascading (HIGH theory)

- **Route:** pick one model up front (RouteLLM arXiv:2406.18665; HybridLLM arXiv:2404.14618).
- **Cascade:** cheap → escalate if quality gate fails (FrugalGPT arXiv:2305.05176; cascade-routing arXiv:2410.10347).
- Cascades need a **calibrated quality signal**; weak judges destroy both quality and cost.

6.3 Anti-patterns (HIGH)

- Silent mid-session model swap → prompt-cache miss + distribution shift (Claude Code warns). Prefer **spawn subagent** on target model.
- Judge weaker than coder without tests.
- Peer agents sharing one dirty tree.
- Silent model remap to hide a budget failure.

6.4 ADE schema direction (`ade.model-profile/v1`)

Bind: `provider · model · allowed_autonomy[] · tool_effect_mask · effort_floor · spend_ceiling · slot_eligibility[] · require_verify · prefer_worktree`.

Router inputs: slot + task risk + spend headroom — not vibes.

7. Architecture for ADE (harness-first)

Eng-goal

- Orchestrator (slots, task queue, claim, heartbeats)
 - Model router (profiles)
 - Planner | Worker | Scout
 - Authorize (ToolEffect · autonomy · leases · PLAN)
 - Execute (FS/shell/MCP · optional worktree)

- Verify (G0-G5 / Automate gate)
- Continuity / Spend ledger

Near-term phases (DEC-A-014 / H-track)

Phase	Ships
H1	SpendGuard honesty (visible caps, reserve/reconcile)
H2	Slot Orchestrator truth (planner/worker bindings)
H3	Model profiles + router
H4	Multi-worker amp + lease conflict UX
H5	Harness gold-set (races, budgets, wrong-slot regressions)
Z?	Zed ACP — maybe , after harness dogfood

8. Synthesis: what “high confidence” implies for builders

1. **Invest in harness depth before editor wrapping.**
2. **Hierarchy + leases > peer chat** for multi-writer coding.
3. **Computational verify before inferential praise.**
4. **Tokenomics:** structure prompts for cache-stable prefixes; progressive disclosure; Continuity thrift; cap rounds / out-tokens / \$ **independently**. (I/O playbook → *ADE Tokenomics* companion.)
5. **Route by role and risk; escalate via subagents, not silent swaps.**
6. **Eval the model-harness pair on a sealed gold set** you own.

9. Selected primary bibliography

Harness / governance

- UNU (2026). *Engineering and Governing the Agent Harness*.
- ContextOS (2026). *Agent Harness* whitepaper.
- arXiv:2605.23950 — *Stop Comparing LLM Agents Without Disclosing the Harness*.

Multi-agent

- arXiv:2601.13295 — CooperBench.
- arXiv:2503.13657 — MAST (*Why Do Multi-Agent LLM Systems Fail?*).
- arXiv:2512.08296 — Scaling agent systems.
- Cursor blogs: scaling agents; cloud agent lessons (first-party).

Verification / evals

- arXiv:2604.05278 — Spec Kit Agents.
- arXiv:2310.06770 — SWE-bench; OpenAI SWE-bench Verified / Pro posts.
- Anthropic: demystifying evals; harness design for long-running apps.
- METR / Cursor: reward hacking reports.

Tokenomics & \$ metering (separate lanes)

- *ADE Tokenomics* — I/O · context · usage (ADE-Tokenomics-I0-Context-Usage.md).
- Anthropic Pricing & Prompt Caching docs (2026) — invoice class.
- OpenAI Pricing & Prompt Caching docs (2026).
- Anthropic: *Effective context engineering for AI agents* (2025).
- Chroma: Context Rot research (2025).
- arXiv:2601.06007 — Don't Break the Cache.

Routing

- arXiv:2305.05176 — FrugalGPT.
- arXiv:2406.18665 — RouteLLM.
- arXiv:2404.14618 — Hybrid LLM.
- arXiv:2410.10347 — Cascade routing.
- Claude Code / Aider / Cursor Router public docs.

ADE internal

- AGENTS.md, DEC-A-010/013/014, ORCHESTRATION_ENG_GOAL_PLAN.md, EFFORT_TURN_BUDGET_PLAN.md, TOKEN_ECONOMICS_RESEARCH.md, crates/workflow leases, crates/agents autonomy/authority/spend.

10. Research provenance

Subagent track	Role
Harness	Definitions, PAE loop, checklist, hype
Tokens	Pricing, cache, failure modes, controls
Multi-agent	CooperBench verification, taxonomy, architecture
Verify/evals	Ladder, gold sets, reward hacking
Routing	Profiles, decision tree, anti-patterns

Parent synthesis + Anthropic pricing + CooperBench abstract spot-checks: 2026-07-23.

Appendix A — Quick reference card for ADE

Concern	Do	Don't
Multi-writer	Leases + worktrees + one merge path	Peer chat on one dirty tree
Done	Verify ladder / Automate gate	Model self-report
Cost	Provider usage + orthogonal caps	chars÷4 billing
Models	Role profiles + subagent escalate	Silent mid-session /model
Product focus	Harness H1–H5	Block on Zed wrap

Appendix B — Spot-check log

Claim	Check	Result
CooperBench ~30% lower Coop vs Solo	arXiv:2601.13295 abstract fetch 2026-07-23	Confirmed wording
Sonnet 4.6 \$3 / \$0.30 cache / \$15 out	docs.anthropic.com pricing fetch	Confirmed
Spec Kit paper exists	arXiv:2604.05278	Confirmed by track D
ToolEffect as cross-vendor standard	Pattern only	Not a formal industry spec — MEDIUM

End of brief. Prices and model SKUs change; re-verify vendor docs before financial planning.